

ReadMe

Code Project

decentralized_inference_cleanup_v1

(Note: less important functions or parameters (e.g., device) could be omitted in readme for more readability)

Classes: *green font*

Functions/methods: *orange font*

Code files: *blue font*

How to run code (Two shell script files + One python file)

Step 1:

command:

`./run_exp-v2.sh`

```
##### vary M (batch_size 8) #####
python trainer_2.py --M 1 --N 30 --L 4 --K 8 --Pmax 15 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_M
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 15 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_M
sleep 5

python trainer_2.py --M 3 --N 30 --L 4 --K 8 --Pmax 15 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_M
sleep 5

python trainer_2.py --M 4 --N 30 --L 4 --K 8 --Pmax 15 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_M
sleep 5

python trainer_2.py --M 5 --N 30 --L 4 --K 8 --Pmax 15 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_M
sleep 5

##### vary Pmax (batch_size 8) #####
python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 5 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 10 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 15 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 20 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 25 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 30 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5

python trainer_2.py --M 2 --N 30 --L 4 --K 8 --Pmax 35 --batch_size 8 --runs 1 --device cuda:0 --out_dir results_batch_8_BS-radius_200_RIS-radius_100_vary_Pmax
sleep 5
```

Explanation:

This shell script executes the [trainer_2.py](#) file for different values of M (number of antennas per access point (AP)) and $P_{\max}$ (power budget per access point (AP)).

Step 2:

command:

`./excel_helper.sh`

```
python excel_helper.py --root results_batch_8_BS-radius_200_RIS-radius_100_vary_M
```

Explanation:

This shell script executes [excel_helper.py](#), creating an excel file which organizes the results from the specified folder.

Step 3:

command:

```
python plot-v2.py
```

Explanation:

Plots the results from the specified excel file as a professional figure.

Explanation of code (Six python files)

(1) trainer_2.py

Overview

Main training and inference module

Classes

- `Trainer`

Trainer

This class

- Trains using centralized approach
- Inferences
 - Using centralized
 - Using decentralized

Initialization:

- Instantiates `MyDataLoader` from `data.py`
 - Calls `BS_RIS_association()`
- Instantiates `node_update` (GNN, DNN models) from `model.py`
- Sets association threshold

```

self.M = M # num of antennas per AP
self.N = N # num of elements per RIS
self.K = K # num of users
self.L = L # num of RISs (Note: In paper, L refers to num of APs)
self.Pmax = 10**((Pt-30)/10)
self.batch_size = batch_size
self.n_iter = 2000
self.dataloader = MyDataLoader(M,N,L,batch_size)
self.dataloader.BS_RIS_association()
self.device = torch.device(device if torch.cuda.is_available() else "cpu")
self.num_of_AP = 5
self.model = node_update(M,N,L,6,self.Pmax,2,64,self.num_of_AP, self.device).to(self.device)
self.min_rate = 1
self.log_interval = 10
self.log_eval_interval = 500

self.training_associate_threshold = 0.1
self.associate_threshold = 0.1
self.dup = False
self.sum_UE = np.zeros((self.num_of_AP))

```

train_batch()

Description	<i>Trains GNN/DNN</i>
Input	None
Output	loss.item(), sum_rate.item(), rate.detach().cpu()

train()

Description	<i>Calls train_batch()</i>
Input	run_id, out_dir, log_dir, test_sample_val, test_sample_final
Output	None

eval():

Description	<i>Inference</i>
Input	test_sample,sigma,itera
Output	np.mean(sum_rate_array_centralized), np.mean(sum_rate_array_centralized_random_phase), np.mean(sum_rate_array_decentralized), np.mean(sum_rate_array_decentralized_random_phase), np.mean(sum_rate_array_centralized_discrete), np.mean(sum_rate_array_decentralized_discrete),

	np.mean(sum_rate_array_centralized_random_phase_discrete), np.mean(sum_rate_array_decentralized_random_phase_discrete)
--	---

(2) model_2.py

Overview

Specifies GNN, DNN architecture

Classes

- `initial_layer` (i.e., preprocessing layer)
- `node_update_layer` (i.e., message passing)
- `BS_readout`
- `RIS_readout_AP` (i.e., for RIS prediction at each AP)
- `RIS_merge`
- `coeff_DNN2` (i.e., for power control)
- `node_update`

node_update

This class chains all of the GNN, DNNs together.

```

for d in range(D):
    update_list.append(node_update_layer(ch*(d+1),M,N,L,ch, device))
self.update_list = nn.ModuleList(update_list)

AP_coeff_NN_list = []
RIS_readout_AP_list = []
for BS_idx in range(self.AP):
    AP_coeff_NN_list.append(coeff_DNN2(M,N,L,Pmax,ch*(D+1)))
    RIS_readout_AP_list.append(RIS_readout_AP(M,N,L,Pmax,ch*(D+1), in_dim2=None))
self.AP_coeff_NN_list = nn.ModuleList(AP_coeff_NN_list)
self.RIS_readout_AP_list = nn.ModuleList(RIS_readout_AP_list)
self.RIS_merge = RIS_merge(N)

self.BS_readout = BS_readout(M,N,L,Pmax,ch*(D+1))

```

“training” in the code means: “centralized”,

```
def forward(self, user_feature, e, user_index, e_dir, training=True, duplicate=False, mean_ue=[]):
    if training:
        #! user_feature.shape = torch.Size([32, 4, 40, 248])
        #! e.shape = torch.Size([32, 4, 40])
        #! user_index.shape = (32, 40)
        #! e_dir.shape = torch.Size([32, 1, 40])
```

else: decentralized

```
else: # decentralized
    batch_size = user_feature[0].shape[0]

    W = torch.zeros((batch_size, 2*self.M, user_feature[0].shape[2]*len(user_feature))).to(self.device)
    theta = torch.zeros((batch_size, user_feature[0].shape[1], self.N, 2)).to(self.device)
```

Code snippet for centralized:

```
##### initial layer & GNN #####
uk, r1 = self.init_user(user_feature_ext, e_ext, e_dir_ext)
for i, _ in enumerate(self.update_list):
    update_layer = self.update_list[i]
    uk, r1 = update_layer(uk, r1, e_ext, A, e_dir_ext)
    uk = uk.to(self.device)
    r1 = r1.to(self.device)

##### power control #####
AP_coeffs = []
for BS_idx in range(self.AP):
    AP_coeffs.append(self.AP_coeff_NN_list[BS_idx](uk))

W_out = self.BS_readout(uk) # unnormalized #! W_out.shape = torch.Size([1, 8, 40])
user = int(user_feature.shape[2]/self.AP) #! user = 8
for BS in range(self.AP):
    BS_served_node = np.array([False]).repeat(user_feature.shape[2]) #! BS_served_node.shape = (40,)
    BS_served_node[BS*user:(BS+1)*user] = actual_served_node[BS*user:(BS+1)*user]
    W[sample, :, BS_served_node] = W_out[:, :, BS_served_node].squeeze(0)

#! W.shape = torch.Size([32, 8, 40])
for BS in range(self.AP):
    BS_W = W[sample, :, BS*user:(BS+1)*user].reshape((1, -1))
    BS_W = F.normalize(BS_W, dim=1, eps=1e-8)*torch.sqrt(self.Pmax * AP_coeffs[BS])
    W[sample, :, BS*user:(BS+1)*user] = BS_W.reshape((2*self.M, -1))

##### RIS readout AP (start) #####
RIS_prior_merge = []
for BS in range(self.AP):
    e_AP = e[sample, :, BS*user:(BS+1)*user].reshape(1, -1) # (1, 32) if user=8, L=4
    R1_custom = self.RIS_readout_AP_list[BS](r1, e_AP)
    RIS_prior_merge.append(R1_custom)

theta_out = self.RIS_merge(RIS_prior_merge) #! theta_out.shape = torch.Size([1, 4, 30, 2])
theta[sample] = theta_out.squeeze() #! theta.shape = torch.Size([32, 4, 30, 2])
```

(3) data.py

Overview

Specifies wireless topology, data loading, and loss calculations.

Classes

- Base_station
- RIS
- MyDataLoader

MyDataLoader

This class

- Specifies and creates the wireless network topology
- Specifies how that data would be loaded
- Computes loss function

Initialization:

Defines BS and RIS locations using *gen_fixed_location()*

```
self.BS_Loc_array = gen_fixed_location(5, self.length*2)
self.BS_array = [Base_station(M,N,L,[self.BS_Loc_array[0]]),Base_station(M,N,L,[self.BS_Loc_array[1]]), \
                 Base_station(M,N,L,[self.BS_Loc_array[2]]), Base_station(M,N,L,[self.BS_Loc_array[3]]), Base_station(M,N,L,[self.BS_Loc_array[4]])]

self.RIS_Loc_array = gen_fixed_location(4, self.length)
self.RIS_array = [RIS(0,self.RIS_Loc_array[0]),RIS(1,self.RIS_Loc_array[1]),\
                 RIS(2,self.RIS_Loc_array[2]),RIS(3,self.RIS_Loc_array[3])]
```

BS_RIS_association()

Description	<i>Associates each Base Station (BS) with all RIS units.</i>
Input	None
Output	None

BS_user_association()

Description	<i>Add UEs to wireless network. Association determined by RSSI.</i>
Input	K, ratio, testing_ratio
Output	None

```
def BS_user_association(self,K,ratio,testing_ratio):
    self.K = K
    loc_user = gen_location(K,self.length)          #! loc_user shape = (8, 2)
    self.user_loc.append(loc_user)

    #Channel Generation and Assignment
    RSSI = np.zeros((self.batch_size,K,len(self.BS_array)))
    for i in range(len(self.BS_array)):              # for each AP
        loc_BS = self.BS_array[i].get_loc()
        loc_RIS = self.BS_array[i].get_RIS_loc()
        L = loc_RIS.shape[0]
        LOS = self.BS_array[i].get_LOS()              #! len(LOS) = 4, #! LOS[0].shape = (30, 4)
        H, channel_bs_user, perfect_H, perfect_bs_user = generate_channel(self.M,self.N,L,K,self.batch_size,LOS,0,loc_RIS,loc_BS,loc_user)
```

load_data()

Description	<i>Generic data loading (to be called)</i>
Input	BS
Output	user_feature, e, user_index, e_dir, user_index_for_testing

gen_training_data()

Description	<i>Calls load_data() for centralized scenario</i>
Input	K, ratio, testing_ratio
Output	user_feature, e, user_index, e_dir, user_index_for_testing

gen_testing_data()

Description	<i>Calls load_data() for decentralized scenario</i>
Input	K, ratio, testing_ratio
Output	user_feature, e, user_index, e_dir

compute_loss()

Description	<i>Calls the cal_loss() function in utils_return_individual_rates.py</i>
Input	W, theta, Pmax
Output	loss, sum_rate, rate

(4) utils_return_indivial_rates.py

Overview

Includes helper functions such as

- functions for topology generation
- channel simulation
- sum-rate calculations

Functions

- `gen_location()`
- `gen_fixed_location()`
- `gen_BS_location()`
- `gen_LOS()`
- `generate_channel()`
- `discrete_mapping()`
- `cal_loss()`

Classes

- `Channel`

Channel

generate_value()

Description	<i>Inserts values for channel object</i>
Input	mean_NLOS, cov_NLOS, LOS
Output	None

large_scale_loss()

Description	<i>Large scale path loss</i>
--------------------	------------------------------

Input	fading_NLOS, exp_NLOS, fading_LOS, exp_LOS, dist
Output	self.mat

(5) excel_helper.py

Overview

Organizes the results from the generated folders to an excel file.

(6) plot-v2.py

Overview

Plots the results from the specified excel file into a professional figure.